

# L07: Manual Event Tracing and Performance Measure Computation

Simulation Without a Black Box

# Outline

Learning Objectives

HW-02 Setup: Pharmacy Queue

Step-by-Step Trace: First Six Events

Simultaneous Events at  $t=18$  and  $t=23$

Area-Under-Curve Method for L

Computing the Five Performance Measures

Python Verification

Key Takeaways

## Learning Objectives

By the end of this lecture you will be able to:

- Set up the HW-02 trace table for a single-server pharmacy queue.
- Execute the first six events of the trace step-by-step.
- Apply the area-under-curve (step-function) method to compute  $L$  and  $L_q$ .
- Derive  $W_q$ ,  $W$ ,  $L$ ,  $L_q$ , and  $\rho$  from trace data.
- Handle simultaneous events at  $t = 18$  and  $t = 23$  using departure-before-arrival.
- Verify trace results with a Python one-liner.

## HW-02 Setup: Single-Server Pharmacy

**System:** One pharmacist, unlimited waiting space, FCFS discipline.

**Given data (10 customers):**

| Customer $i$       | 1 | 2 | 3 | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|--------------------|---|---|---|----|----|----|----|----|----|----|
| Arrival time $a_i$ | 0 | 5 | 9 | 14 | 18 | 23 | 29 | 35 | 40 | 46 |
| Service time $s_i$ | 8 | 6 | 4 | 7  | 3  | 9  | 5  | 6  | 4  | 7  |

**Trace table columns needed:**

- Event time  $t$ , event type,  $N_q(t)$ , server status  $B(t)$ , service-begin time  $b_i$ , departure time  $d_i$ , wait  $W_q^{(i)} = b_i - a_i$ , sojourn  $W^{(i)} = d_i - a_i$ .

## Trace: Events 1–6

| Evt | Type         | $t$ | $N_q$ | $B$ | Action                                        | $W_q^{(i)}$ |
|-----|--------------|-----|-------|-----|-----------------------------------------------|-------------|
| 1   | Arrival (C1) | 0   | 0     | 1   | C1 seizes server; svc-end@8                   | —           |
| 2   | Arrival (C2) | 5   | 1     | 1   | C2 joins queue                                | —           |
| 3   | Svc-end (C1) | 8   | 0     | 1   | C1 departs ( $W_q=0$ ); C2 begins; svc-end@14 | 0           |
| 4   | Arrival (C3) | 9   | 1     | 1   | C3 joins queue                                | —           |
| 5   | Svc-end (C2) | 14  | 0     | 1   | C2 departs ( $W_q=3$ ); C3 begins; svc-end@18 | 3           |
| 6   | Arrival (C4) | 14  | 1     | 1   | C4 joins queue                                | —           |

*Event 3 (svc-end) and Event 4 (arrival) are not simultaneous here. Continue the table for all 10 customers as HW-02.*

## Simultaneous Events: $t = 18$ and $t = 23$

**At  $t = 18$ :** C3's service ends AND C5 arrives simultaneously.

### Departure-first (correct):

1. C3 departs; C4 begins service; schedule `svc-end@25`.
2. C5 arrives;  $N_q = 1$ ; joins queue.

Result:  $W_q^{(5)} = b_5 - 18 > 0$ .

### Arrival-first (incorrect):

1. C5 arrives; sees C3 in service;  $N_q = 2$ .
2. C3 departs; C4 begins;  $N_q = 1$ .

Result: same  $W_q^{(5)}$ , but queue length spike differs.  $L_q$  is slightly larger than it should be.

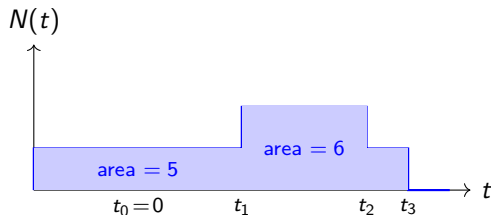
### Apply consistently

Departure-before-arrival at  $t = 23$  as well (C4 `svc-end`, C6 arrival). The convention must be uniform throughout the trace.

## Area-Under-Curve Method for $L$ and $L_q$

$N(t)$  is a step function that changes only at events. Time-average  $L$ :

$$L = \frac{1}{T} \int_0^T N(t) dt = \frac{1}{T} \sum_k N(t_k) \cdot (t_{k+1} - t_k)$$



**Cumulative area column in trace:**

| $t_k$ | $N(t_k)$ | $\Delta t$ | Area |
|-------|----------|------------|------|
| 0     | 1        | 5          | 5    |
| 5     | 2        | 3          | 6    |
| 8     | 1        | 1          | 1    |
| 9     | 2        | 5          | 10   |
| ...   |          |            |      |

$$L = \sum \text{Area} / T$$

## Computing $W_q$ , $W$ , $L$ , $L_q$ , $\rho$ from Trace Data

| Measure | Formula from trace                       | Notes                           |
|---------|------------------------------------------|---------------------------------|
| $W_q$   | $\frac{1}{n} \sum_{i=1}^n (b_i - a_i)$   | Average queue wait per customer |
| $W$     | $\frac{1}{n} \sum_{i=1}^n (d_i - a_i)$   | Average sojourn per customer    |
| $L$     | $\frac{1}{T} \sum_k N(t_k) \Delta t_k$   | Time-average system occupancy   |
| $L_q$   | $\frac{1}{T} \sum_k N_q(t_k) \Delta t_k$ | Time-average queue length       |
| $\rho$  | $\frac{1}{T} \int_0^T B(t) dt$           | Fraction of time server is busy |

### Sanity check

Little's Law must hold:  $L \approx \hat{\lambda} W$  and  $L_q \approx \hat{\lambda} W_q$ , where  $\hat{\lambda} = n/T$  is the observed throughput rate.



## Python One-Liner to Verify the Trace

Once you have completed the manual trace, verify with:

```
from simdes.models.queues import trace_single_server

arrivals      = [0, 5, 9, 14, 18, 23, 29, 35, 40, 46]
service_times = [8, 6, 4, 7, 3, 9, 5, 6, 4, 7]

results = trace_single_server(arrivals, service_times)
print(results)    # W, Wq, L, Lq, rho
```

### Verification strategy

If your hand trace disagrees with the code, the disagreement is your first verification finding. Diagnose it before submitting HW-02: is the error in the trace logic, the code, or the tie-breaking convention?

*Manual tracing is the gold standard for verification of toy cases. The code should match the trace, not the other way around.*

## Key Takeaways

- A complete trace table has:  $t$ , event type,  $N_q$ ,  $B$ ,  $b_i$ ,  $d_i$ ,  $W_q^{(i)}$ , area.
- Use the departure-before-arrival convention at every tie — consistently.
- $L$  and  $L_q$  are computed by summing  $N(t_k) \cdot \Delta t_k$  over all intervals and dividing by  $T$ .
- $W_q$  and  $W$  are customer-average statistics;  $L$  and  $L_q$  are time-average statistics.
- Little's Law is the fast sanity check:  $L = \hat{\lambda} W$  must hold to two decimal places.
- `trace_single_server()` verifies the trace — use it to diagnose, not to replace.

**Next lecture:** Little's Law — proof, decomposition, and applications (L08).